STC89C52 Simulator: Interactive UI & Complete Working Guide

STC89C52 Microcontroller Multi-Process Simulator - Student Study Guide

Visual Operations Manual, Panel Breakdown & Step-by-Step State Guide

Executive Overview: What Is This Simulator?

The **STC89C52 Multi-Process Simulator** is a virtual, cycle-accurate software recreation of the classic **Intel 8051-compatible STC89C52 8-bit microcontroller**.

Instead of requiring physical silicon chips, breadboards, and hardware programmers, this software emulates the internal hardware registers, 256 bytes of internal RAM, ALU arithmetic flags, the hardware call stack, and an on-chip **Circular FIFO Queue buffer**.

To mirror how real microcontrollers connect to host computers (via serial UART/SPI cables), this simulator is built as **three separate operating system processes** talking over network cables (POSIX TCP Sockets):

1. `CoreProcess` **(Port 5000):** The CPU execution brain. It fetches opcodes, computes math, changes flags, and holds RAM.
2. `UIProcess` **(Port 8080 & 8000):** The visual dashboard. It runs an embedded web server that you open in any browser.
3. `LoggerProcess` **(Port 5001):** The black-box flight recorder. It writes every instruction and queue event to `simulator.log` on disk.

1. The Control Buttons (Top Navigation Bar)

At the very top of the dashboard is your **remote control panel**:

| Button | Exact Technical Action | What Happens to the Virtual Chip |

| :--- | :--- | :--- |

| `[ Reset ]` | Sends `RESET` command to CoreProcess | Restores Program Counter $PC = 0x0000$, Stack Pointer $SP = 0x07$, Accumulator $A = 0x00$, clears all PSW flags, and drains the FIFO queue. |

| `[ Load Hex ]` | Sends `LOAD <hex>` to CoreProcess | Takes machine code bytes from Card 1, parses out comments, burns the bytes into RAM starting at address `0x0000`, and resets the CPU ready to execute. |

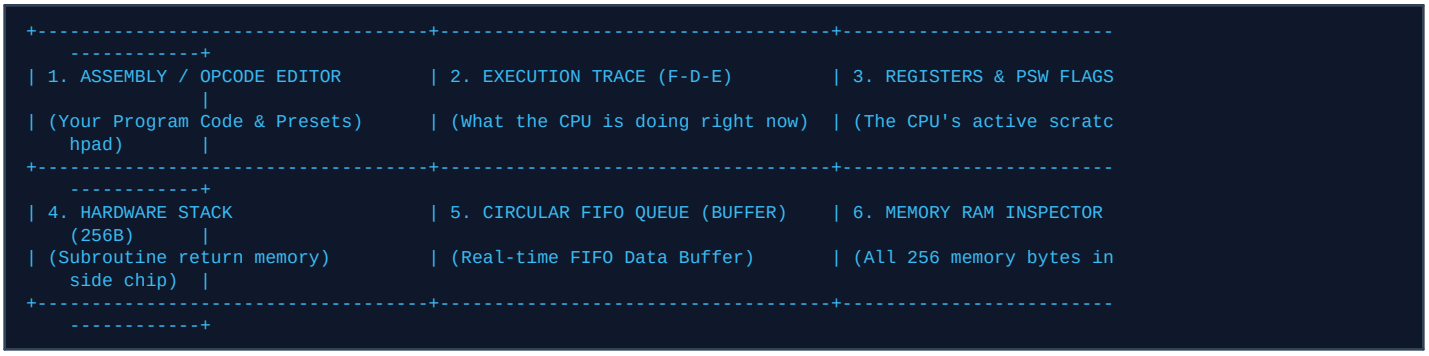
| `[ Step (F-D-E) ]` | Sends `STEP` command to CoreProcess | **The most important button.** Executes **one single instruction cycle** (Fetch -> Decode -> Execute) and immediately halts. Allows frame-by-frame debugging. |

| `[ Run Continuous ]` | Sends `RUN` command to CoreProcess | Instructs the CPU to execute instructions in a rapid loop without pausing until it hits opcode `0xFF` (`HALT`). |

| `[ Stop / Halt ]` | Sends `STOP` command to CoreProcess | Emergency brake. Halts CPU execution if trapped in an unintentional infinite loop. |

| `[ Refresh ]` | Sends `STATE` command to CoreProcess | Queries the CPU over port 5000 and redraws all 6 panels with the latest hardware numbers. |

2. The 6 Dashboard Panels (What Does Each Box Mean?)



Card 1: Assembly / Hex Opcode Editor (Program Input)

- **Purpose:** Where the user types or pastes machine code bytes in hexadecimal format.
- **Starting Code Preset Buttons:**
- `[ Preset 1: Math & Flags ]`: Loads addition with carry overflow ($254 + 3 = 257 ightarrow A=1, CY=1$).
- `[ Preset 2: FIFO Queue ]`: Loads assembly program to enqueue 10, 30, 5, then dequeue them.
- `[ Preset 3: Logic (ANL) ]`: Loads bitwise AND masking program ($0xAB ext{ AND } 0x0F ightarrow 0x0B$).
- `[ Preset 4: Jump (SJMP) ]`: Loads relative branch program that skips dummy instructions.
- **Assembly Comment Support:** Lines starting with `;` or `//` are automatically ignored by the parser, so human comments can be written alongside hex bytes.
- **Opcode Quick Reference:** Displays the 8 supported instructions directly inside the card.

Card 2: Execution Trace (The CPU's Internal Pipeline)

Whenever you click `[ Step ]`, this panel displays the **3-phase instruction heartbeat**:

- **Phase 1: Fetch:** The byte read from memory at the current Program Counter (e.g. `PC=0x0000 -> Opcode 0x74`).
- **Phase 2: Decode:** The mnemonic identified by the CPU (e.g. `MOV A, #data [0x74]`).
- **Phase 3: Execute:** The mathematical operation or data transfer performed (e.g. `A : 00 -> 0A`).
- **State Transition:** Shows before-and-after values for the Accumulator and Carry flag (e.g. `ACC: 0x00 -> 0x0A`, `CY: false -> false`).

Card 3: Registers & PSW Flags (The Active CPU Workstation)

- `PC` **(Program Counter):** 16-bit register holding the memory address of the next instruction to execute.
- `SP` **(Stack Pointer):** Points to the current top of the internal call stack. Resets to `0x07`.
- `ACC` **(Accumulator A):** The primary 8-bit math and logic register where arithmetic occurs.
- `B`**:** Secondary register used for multiplication and division operations.
- `R0` **through** `R7`**:** The 8 working registers of the currently active Register Bank (Bank 0).
- **The PSW Flags (Program Status Word):** 4 status lightbulbs that turn green (1) or dark (0):
- `CY` **(Carry Flag):** Turns green if an addition exceeds 8 bits (> 255).
- `AC` **(Auxiliary Carry):** Turns green if lower 4 bits carry into bit 4 (used for BCD/clock math).
- `OV` **(Overflow Flag):** Turns green if signed two's complement arithmetic overflows (e.g. $-128 + -128 = 0$).
- `P` **(Parity Flag):** Turns green if the number of set binary 1s in Accumulator A is odd.

Card 4: Hardware Stack (Call Stack Subsystem)

- **Top of Stack:** Shows the byte stored at `RAM[SP]`.
- **Stack Depth:** Shows how many items are pushed above the reset baseline.
- **Why SP Resets to** `0x07`**:** In the 8051 architecture, RAM addresses `0x00` through `0x07` are physically occupied by Register Bank 0 (`R0-R7`). When an item is pushed, `SP` pre-increments (`++SP`). Starting at `0x07` guarantees the first pushed byte lands safely at address `0x08`, preventing register corruption!

Card 5: Circular FIFO Queue (Real-Time Buffer Subsystem)

- **Purpose:** Demonstrates on-chip buffered data streaming (First-In, First-Out).
- `Size`**:** Current number of elements waiting in the buffer.
- `Capacity: 16`**:** Maximum number of items the buffer can hold without overflow.
- `Front`**:** Index where the next `DEQUEUE` will read from.
- `Rear`**:** Index where the next `ENQUEUE` will write into.
- `Elements`**:** Live visual array of queued numbers (e.g. `[ 0x0A, 0x1E, 0x05 ]`).
- **Circular Modulo Wrap:** When `rear` reaches index 15, it smoothly wraps back to index 0 using `(rear + 1) % 16`.

Card 6: Memory RAM Inspector (256-Byte On-Chip RAM)

- Displays all 256 bytes of internal microcontroller RAM in a formatted $16 imes 16$ hexadecimal grid.
- Address labels on the left (`0x00:`, `0x10:`, `0x20:`, ..., `0xF0:`).
- Column headers on top (`+0`, `+1`, ..., `+F`).
- When you click `[ Load Hex ]`, you can visually confirm your program bytes stored at address `0x00:`.

3. Step-by-Step Walkthrough: What Changes When You Use It?

Walkthrough 1: Running the FIFO Queue Test (Preset 2)

1. **Click** `[ Preset 2: FIFO Queue ]`**:**
- The editor in Card 1 loads the queue test assembly program:

```
74 0A E0 74 1E E0 74 05 E0 D0 D0 D0 FF
```

- The CPU automatically resets: `PC = 0x0000`, `ACC = 0x00`, Queue Size = `0`.
2. **Click** `[ Step ]` **(1st time):**
- **Instruction:** `MOV A, #10` (`0x74 0x0A`)
- **What Changes:** `PC` moves `0x0000 -> 0x0002`. Accumulator `ACC` changes from `0x00` to `0x0A (10)`. Parity `P` turns green.
3. **Click** `[ Step ]` **(2nd time):**
- **Instruction:** `ENQUEUE A` (`0xE0`)
- **What Changes:** `PC` moves to `0x0003`. In Card 5, **Queue Size becomes** `1`, and **Elements shows** `[ 0x0A ]`.
4. **Click** `[ Step ]` **(3rd and 4th times):**
- `MOV A, #30` loads `0x1E (30)` into `ACC`.
- `ENQUEUE A` pushes `0x1E` into the queue.
- In Card 5, **Queue Size becomes** `2`, and **Elements shows** `[ 0x0A, 0x1E ]`.
5. **Click** `[ Step ]` **(5th and 6th times):**
- `MOV A, #5` loads `0x05 (5)` into `ACC`.
- `ENQUEUE A` pushes `0x05` into the queue.
- In Card 5, **Queue Size becomes** `3`, and **Elements shows** `[ 0x0A, 0x1E, 0x05 ]`.
6. **Click** `[ Step ]` **(7th time - First Dequeue!):**
- **Instruction:** `DEQUEUE A` (`0xD0`)
- **What Changes:** The oldest item in line (`10`) is extracted. `ACC` becomes `0x0A (10)`. Queue Size drops to `2`, leaving `[ 0x1E, 0x05 ]`.
7. **Click** `[ Run Continuous ]`**:**
- The CPU executes the remaining dequeues at full speed: `30` is dequeued, then `5` is dequeued.
- Queue becomes completely empty (`Size: 0`, `Elements: --`).
- The CPU executes `0xFF` (`HALT`).
- The status bar turns green: `STATUS: Halted`.

Walkthrough 2: Arithmetic Carry & Half-Carry (Preset 1)

1. **Click** `[ Preset 1: Math & Flags ]`**:**
- Program: `74 FE 24 03 04 ...` (Loads 254, adds 3, increments).
2. **Click** `[ Step ]` **once (**`MOV A, #254`**):**
- Accumulator `ACC` becomes `0xFE (254)`. `CY = 0`.
3. **Click** `[ Step ]` **second time (**`ADD A, #3`**):**
- Math calculation: 254 + 3 = 257.
- Because 8 bits only hold 0 to 255: 257 mod 256 = 1 -> **ACC = 0x01**.
- **Flag Changes in Card 3:**
- `CY` (Carry) turns **GREEN** (Sum exceeded 255).
- `AC` (Aux Carry) turns **GREEN** (Lower nibble 0x0E + 0x03 = 0x11 > 15).
4. **Click** `[ Step ]` **third time (**`INC A`**):**
- Accumulator increments to `0x02`.
- **Notice:** `CY` remains green! Per 8051 specifications, `INC` does **not** modify the Carry flag.

4. Summary Table of Supported Opcodes

Hex Opcode	Mnemonic	Operands	What It Does to Hardware	Flags Modified

`00`	`NOP`	None	Consumes 1 machine cycle, advances PC, does not alter data	None
`74 <d>`	`MOV A, #data`	Immediate 8-bit	Loads immediate constant byte into Accumulator A	Parity `P`
`24 <d>`	`ADD A, #data`	Immediate 8-bit	Adds immediate constant byte to Accumulator A	`CY`, `AC`, `OV`, `P`
`04`	`INC A`	None	Increments Accumulator A by 1 modulo 256	Parity `P` (`CY` unchanged!)
`54 <d>`	`ANL A, #data`	Immediate 8-bit	Performs bitwise logical AND between A and operand	Parity `P`
`80 <r>`	`SJMP rel`	Signed 8-bit	Adds signed relative offset (-128 to +127) to PC	None
`E0`	`ENQUEUE A`	None	Inserts current value of Accumulator A into FIFO queue	None
`D0`	`DEQUEUE A`	None	Extracts oldest item from FIFO queue into Accumulator A	Parity `P`
`FF`	`HALT`	None	Sets `isHalt = true`, stopping further CPU execution	None

5. Troubleshooting & Frequently Asked Questions

Q1: "What does it mean if the status indicator is RED?"

Answer: *It means `UIProcess` cannot find `CoreProcess` on TCP port 5000. Start `CoreProcess` in a terminal window (`java -cp src CoreProcess`), and the indicator will immediately turn green.*

Q2: "Can I type my own custom hex bytes in Card 1?"

Answer: *Yes! Type any valid opcodes separated by spaces (e.g. `74 55 04 04 FF`). You can even add assembly comments like `74 55 ; load 55`. Then click `[ Load Hex ]` and click `[ Step ]` to watch your custom code execute!*

Q3: "Where are execution logs physically saved on my computer?"

Answer: *Every step, reset, and queue operation is transmitted over TCP port 5001 to `LoggerProcess`, which commits each entry directly to `simulator.log` in your project folder.*